ADVANCED PROGRAMMING LANGUAGES

Spring 2026, CM 0632. MSc in Computer Science and Information Technology, Ca' Foscari University of Venice

Higher-order Challenge I — Feedback

Challenge 1: Fold it

To solve the challenge one should identify the common pattern among the various definitions. If you try and solve them by direct recursion, you see that they all have the same structure, i.e. they accumulate (or *fold*) the elements of a list on a single value,one element at the time, by applying a function to the head of the list and the folding of the rest of the list.

In each definition you choose the function to apply and what to return when folding the function on no element

- in `mySum` the function is `(+)` , and the element to return is 0, the identity for `(+)`
- in `myProd` the function is `(*)` and the element to return is 1, the identity for `(*)`
- in `myLength` the function is `add-one` and the element to return is 1, the identity for addition
- in `myConcat` the function is (++) and the element to return is the empty list, the identity for `(++)`

```
foldr :: (a -> b -> b) -> b -> [a] -> b
foldr f b []     = b
foldr f b (a:as) = f a (foldr f b as)

mySum     = foldr (+) 0
myProduct = foldr (*) 1
myLength  = foldr (\_ n -> n + 1) 0
myConcat  = foldr (++) []
```

Note: some of you tried the following

```
myLength = foldr (+1) 0
```

with the idea that at each step we add one to the currently accumulated value. This works intuitively by clashes with the type of the function that `foldr` expects for `f`. To see why, try

```
:type (+1)  -- Num a => a -> a
```

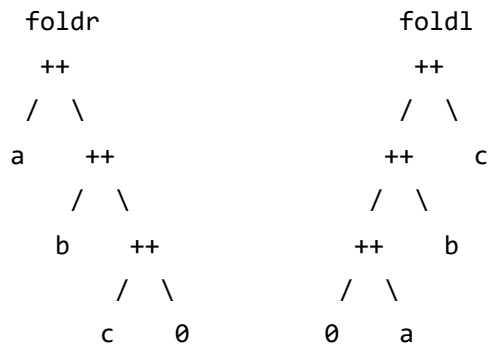
which is not an instance of the expected type `(a -> b -> b)

Challenge 2: Left or right?

```
ex1 = foldr (\x acc -> "(" ++ x ++ "+" ++ acc ++ ")") "0" ["a","b","c"]
-- "(a+(b+(c+0)))"

ex2 = foldl (\acc x -> "(" ++ acc ++ "+" ++ x ++ ")") "0" ["a","b","c"]
-- "(((0+a)+b)+c)"
```

The trees:



Challenge 3: The universal fold

```haskell
myAny :: (a -> Bool) -> [a] -> Bool
myAny p = foldr (\x acc -> p x || acc) False

myAll :: (a -> Bool) -> [a] -> Bool
myAll p = foldr (\x acc -> p x && acc) True

myTakeWhile :: (a -> Bool) -> [a] -> [a]
myTakeWhile p = foldr (\x acc -> if p x then x : acc else []) []
```

In `myTakeWhile` `foldr` effectively stops as expected, in Haskell. This is due to the Haskell's lazy / call by name evaluation:

- The key is that `f x (foldr f b xs)` does not evaluate `foldr f b xs` eagerly: if `f` ignores its second argument, the the argument is never evaluated and the recursion stops.
- This is precisely what happens in `myTakeWhile` : when `p x` is false, the lambda returns `[]` without ever inspecting `acc` , which is bound to `foldr ... xs` . As a result the recursive call is disregarded as soon as `p x` returns false.

The result would be different in a language with strict / call by value like Scalal

Challenge 4: Pipeline

Key decisions:

- *What should `pipe []` be?*
  The identity function `id` . An empty pipeline does nothing to its input — `id` is the identity for function composition, just as `0` is the identity for `(+)` .
- *What operator combines two functions into one?*
  The composition operator `(.)` .
- *Should you fold from the right or from the left?*
  The natural choice is from the right as we expect `f1 (f2 (f3 x))` , which is `f1 . f2 . f3` . `foldr` produces exactly this shape.

```haskell
pipe :: [a -> a] -> a -> a
pipe = foldr (.) id
```

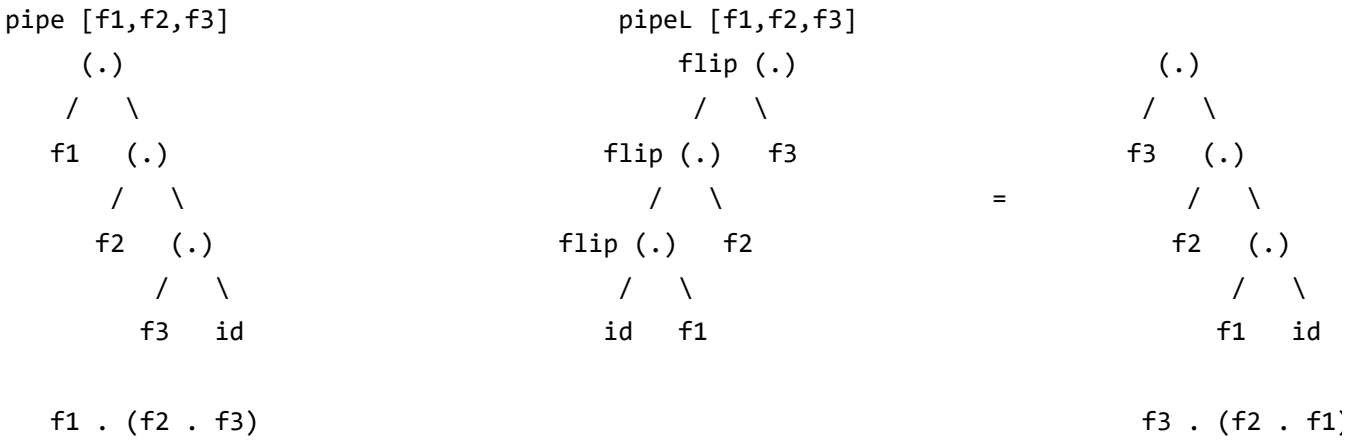
Trace: `pipe [(+1), (*2), (+3)] 4` gives `((+1).(*2).(+3)) 4 = (+1) ((*2) ((+3) 4)) = 15`.

As to the variant, we may define it as follows:

```
pipeL :: [a -> a] -> a -> a
pipeL = foldl (flip (.)) id   -- flip exchanges the order of the two arguments of (.)
```

Trace: `pipeL [(+1), (*2), (+3)] 4` gives `((+3).(*2).(+1)) 4 = (+3) ((*2) ((+1) 4)) = 13`.

Looking at the trees, we have:



Some of you notices that `(foldr (.) id) = (foldl (.) id)`. That is true, and is a consequence of `(.)` being *associative*, which means that for parenthesization does not matter: in fact `f1 . (f2 . f3) = (f1 . f2) . f3`

Challenge 5: Fold fusion

Part 1.

- `resultA = sum (map (*2) [1..n])`. The `map (*2)` produces `[2, 4, 6, ..., 2n]`, and `sum` adds them up.
- `resultB = foldr (\x acc -> 2*x + acc) 0 [1..n]` computes `2*1 + (2*2 + (... + (2*n + 0)...))` — the same sum.
- So `resultA == resultB`.

Part 2. Version A traverses the list *twice*: once for `map`, once for `sum`. Version B traverses it *once*: the doubling and the addition happen in the same pass.

Part 3. First observe that:

- `sum = foldr (+) 0`
- `\x acc -> 2*x + acc = (+).(*2)`

Then, taking

- `f = (*2)`
- `f = (+)`
- `b = 0`

the *fold-map fusion* gets instantiated as follows:

```
(foldr (+) 0) . (map (*2)) = foldr ((+).(*2)) 0
```